武汉大学 国家网络安全学院
SCHOOL OF CYBER SCIENCE AND ENGINEERING·WHU

信息系统安全

（Information System Security）

武汉大学
王鹃

jwang@whu.edu.cn

目录

- 信息系统安全概述
- 软件安全与恶意代码
- 渗透测试方法及工具
- 模糊测试与漏洞挖掘
- 用户身份认证机制
- 访问控制模型
- 操作系统基础安全机制
- 硬件安全与可信计算
- 系统风险评估方法与工具
- 信息系统安全研究新方向

第五章 用户身份认证机制

LISS
空天信息安全与可信计算
教育部重点实验室

目录

1 身份认证概述

2 基于口令的身份认证

3 基于智能卡的身份认证

4 局域网身份认证-Kerberos

5 统一身份认证-PAM

1、身份认证概述

在现实生活中，我们个人的身份主要是通过各种证件来确认的，比如：身份证、户口本等。

认证是对网络中的主体进行验证的过程，用户必须提供他是谁的证明，他是某个雇员，某个组织的代理、某个软件过程（如交易过程）。

认证(Authentication)：是证明一个对象的身份的过程。

1、身份认证概述

- **问题的提出：**

  身份欺诈

- **鉴别需求：**

  某一成员（声称者）提交一 个主体的身份并声称它是那个主体。

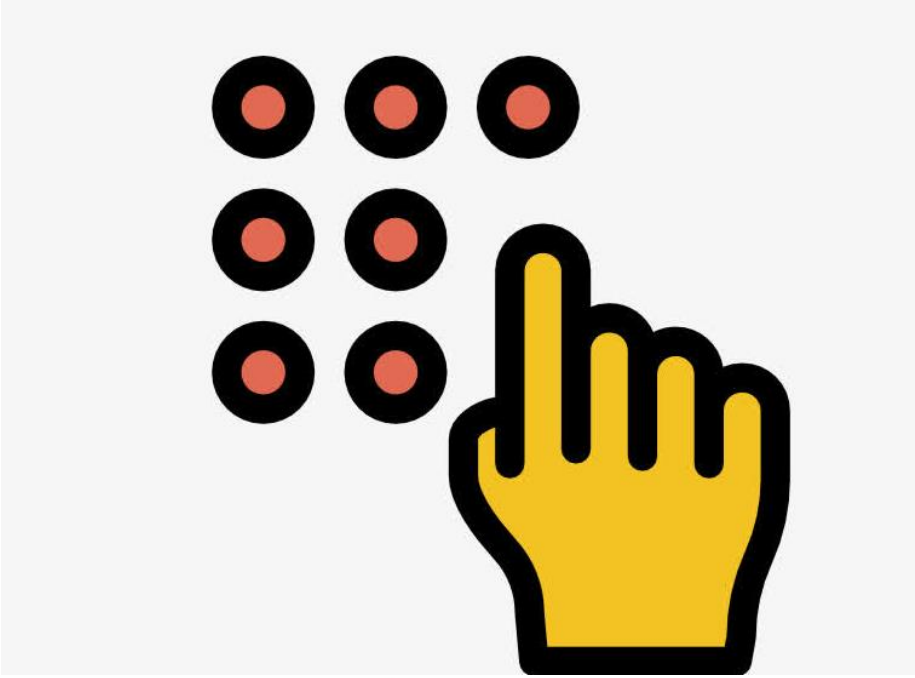
- **鉴别目的：**

  使别的成员（验证者）获得对声称者所声称的事实的信任。

1、身份认证概述

通常有三种方法验证主体身份：

1）只有该主体了解的秘密，如口令、密钥



1、身份认证概述

2）主体携带的物品，如智能卡和令牌卡

1、身份认证概述

3）只有该主体具有的独一无二的特征或能力，如指纹、声音、视网膜图或签字等

单独使用一种方法进行认证是不充分的

1、身份认证概述

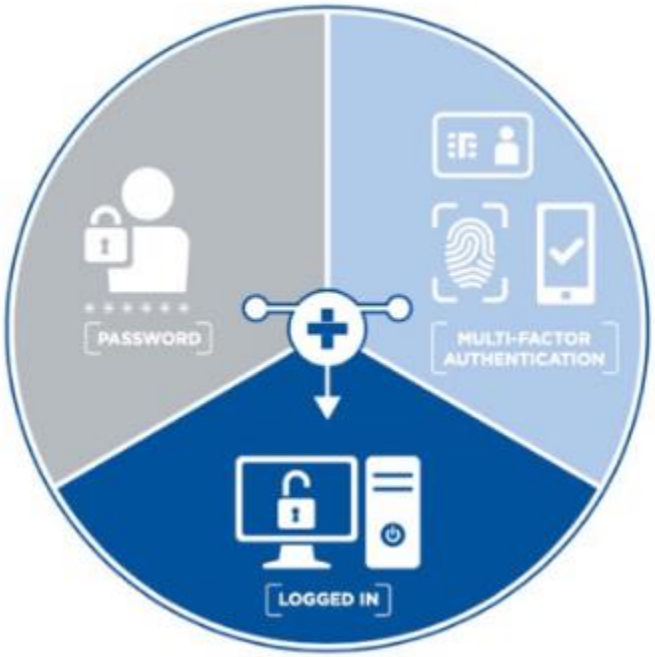
一个成熟的身份认证系统应该具有以下特征：

1、验证者正确识别对方的概率极大。
2、攻击者伪装以骗取信任的成功率极小。
3、通过重放攻击进行欺骗和伪装的成功率极小。
4、实现身份认证的算法计算量足够小。
5、实现身份认证所需的通信量足够小。
6、秘密参数能够安全存储。
7、对可信第三方的无条件信任。
8、可证明安全性。

1、身份认证概述

计算机系统采取的身份认证方法有很多：

- 口令认证
- 智能卡认证
- 基于生物特征的认证
- 双因素认证
- 基于源地址的认证
- 数字证书和安全协议等



1、身份认证概述

身份认证系统架构包含三项主要组成元件：

（1）认证服务器(Authentication Server)：

负责进行使用者身份认证的工作，服务器上存放使用者的私有密钥、认证方式及其他使用者认证的信息。

1、身份认证概述

（2）认证系统用户端软件(Authentication Client Software)：

认证系统用户端通常都是需要进行登陆(login)的设备或系统，在这些设备及系统中必须具备可以与认证服务器协同运作的认证协定。

1、身份认证概述

（3）认证设备(Authenticator)：

认证设备是使用者用来产生或计算密码的软硬件设备。

2、基于口令的身份认证

用户名/口令认证技术：最简单、最普遍的身份识别技术，如：各类系统的登录等。

口令具有共享秘密的属性，是相互约定的代码，只有用户和系统知道。例如，用户把他的用户名和口令送服务器，服务器操作系统鉴别该用户。

2、基于口令的身份认证

口令有时由用户选择，有时由系统分配。通常情况下，用户先输入某种标志信息，比如用户名和ID号，然后系统询问用户口令，若口令与用户文件中的相匹配，用户即可进入访问。

口令有多种，如一次性口令；还有基于时间的口令

2、基于口令的身份认证

- 安全口令与不安全口令：

UNIX系统口令密码都是用8位(新的是13位)DES算法进行加密的，即有效密码只有前8位，所以一味靠密码的长度是不可以的。安全的口令要求：

1) 位数>6位。

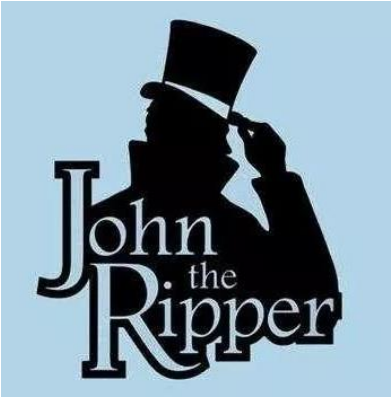
2) 大小写字母混合。

3)字母与数字混合。

4) 口令有字母、数字以外的符号。

2、基于口令的身份认证

不安全的口令则有如下几种情况：



(1)使用用户名（帐号）作为口令。

(2)使用用户名（帐号）的变换形式作为口令。将用户名颠倒或者加前后缀作为口令，比如著名的黑客软件John，如果你的用户名是fool，那么它在尝试使用fool作为口令之后，还会试着使用诸如fool123、loof、loof123、lofo等作为口令，只要是你想得到的变换方法，John也会想得到。

2、基于口令的身份认证

(3)使用自己或者亲友的生日作为口令。这种口令很脆弱，因为这样虽然往往可以得到一个6位或者8位的口令，但实际上可能的表达方式只有100×12×31=37200种。

(4)使用常用的英文单词作为口令。这种方法比前几种方法要安全一些。但一般用户选择的英文单词几乎都落在黑客的字典库里。

(5)使用5位或5位以下的字符作为口令。

2、基于口令的身份认证

十大不安全密码	中国人最常用密码
1. 123456	1. abc123
2. psaaword	2. 123456
3. 12345678	3. xiaoming
4. qwerty	4. 12345678
5. abc123	5. iloveyou
6. 123456789	6. admin
7. 111111	7. qq123456
8. 1234567	8. taobao
9. iloveyou	9. root
10. adobe123	10. wang1234

2、基于口令的身份认证

加强口令安全的措施：

A、禁止使用缺省口令。

B、定期更换口令。

C、保持口令历史记录，使用户不能循环使用旧口令。

D、用口令破解程序测试口令。

2、基于口令的身份认证

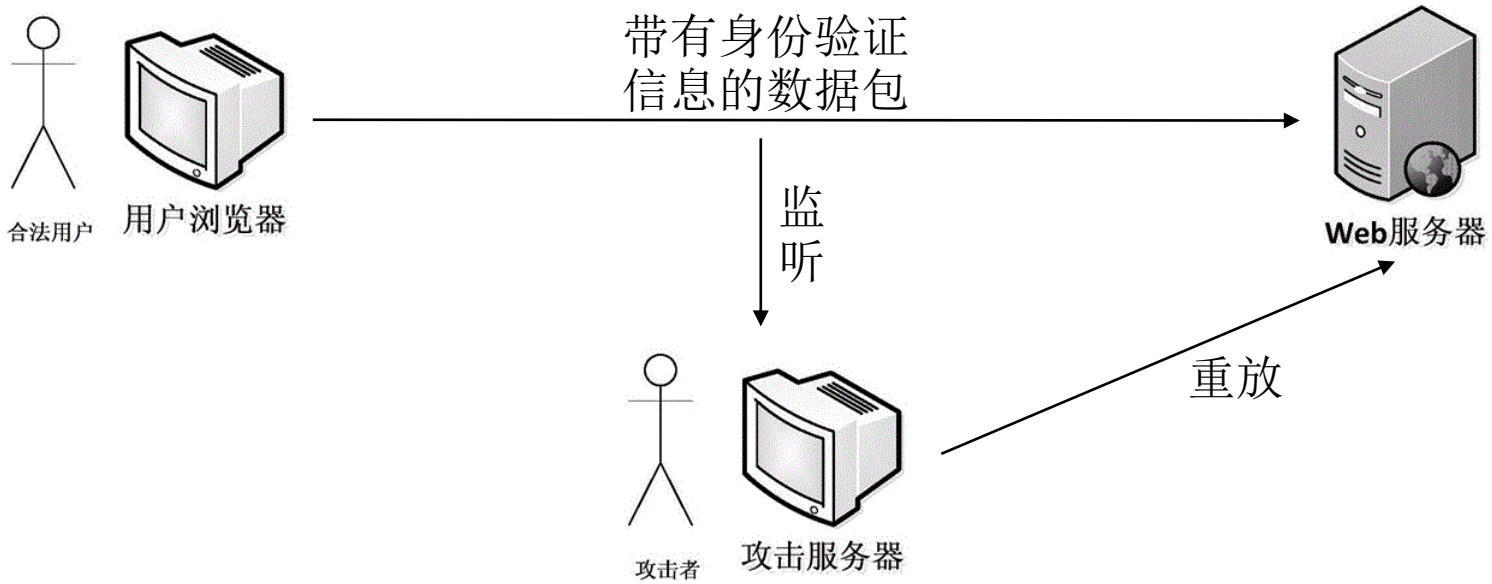
计算资源依靠口令的方式来保护具有脆弱性

- 口令攻击的种类：

（1）网络数据流窃听。由于认证信息要通过网络传递，并且很多认证系统的口令是未经加密的明文，攻击者通过窃听网络数据，就很容易分辨出某种特定系统的认证数据，并提取出用户名和口令。口令被盗意味着用户在这台机器上的一切信息将全部丧失，并且危及他人信息安全，计算机只认口令不认人。最常见的是电子邮件被非法截获。

2、基于口令的身份认证

（2）认证信息截取/重放(Record/Replay)。 有的系统会将认证信息进行简单加密后进行传输，如果攻击者无法用第一种方式推算出密码，可以使用截取/重放方式。攻击者仍可以采用离线方式对口令密文实施字典攻击；



2、基于口令的身份认证

对付重放的方法有：

1.在认证交换中使用一个序数来给每一个消息报文编号，仅当收到的消息序号合法时才接受之；

2.使用时间戳 (A接受一个新消息仅当该消息包含一个 A认为是足够接近 A所知道的时间戳);

3.询问 /应答方式 (A期望从 B获得一个新消息，则先发给 B一个临时值，并要求后续从 B收到的消息包含正确的这个临时值)

2、基于口令的身份认证

（3）字典攻击：由于多数用户习惯使用有意义的单词或数字作为密码，某些攻击者会使用字典中的单词来尝试用户的密码。所以大多数系统都建议用户在口令中加入特殊字符，以增加口令的安全性。

字典攻击	暴力攻击
Trying apple : failed	Trying aaaa : failed
Trying blueberry : failed	Trying aaab : failed
Trying justinbeiber : failed	Trying aaac : failed
...	...
Trying letmein : failed	Trying acdb : failed
Trying s3cr3t : success!	Trying acdc : success!

2、基于口令的身份认证

（4）穷举尝试(Brute Force)： 这是一种特殊的字典攻击，它使用字符串的全集作为字典。如果用户的密码较短，很容易被穷举出来，因而很多系统都建议用户使用长口令。

（5）窥探： 攻击者利用与被攻击系统接近的机会，安装监视器或亲自窥探合法用户输入口令的过程，以得到口令。

2、基于口令的身份认证

（6）社交工程：攻击者冒充合法用户发送邮件或打电话给管理人员，以骗取用户口令。比如，在终端上发现如下信息：

Please enter your user name to logon:

Your password:

这很可能是一个模仿登录信息的特洛伊木马程序，他会记录口令，然后传给入侵者。

（7）垃圾搜索：攻击者通过搜索被攻击者的废弃物，得到与攻击系统有关的信息，如果用户将口令写在纸上又随便丢弃，则很容易成为垃圾搜索的攻击对象。

2、基于口令的身份认证

口令猜中概率公式： $P=L \bullet R/S$

L：口令生命周期 R：进攻者单位时间内猜测不同口令次数

S：所有可能口令的数目。

为降低猜中的概率：

1. 减少口令使用寿命，即提高口令更换的频率；

2. 降低进攻者单位时间内猜测尝试口令的次数；

3. 增加可能口令的数目，即提高口令的字符个数。

Windows下LM Hash值生成原理

① 假设明文口令是 *“Welcome”*，首先全部转换成大写 *“WELCOME”*，再做将口令字符串大写转后后的字符串变换成二进制串：

“WELCOME” -> 57454C434F4D4500000000000000

② 然后切割成两组7字节的数据，分别经*str_to_key*（）函数处理得到两组8字节数据：

57454C434F4D45 -str_to_key()-> 56A25288347A348A

00000000000000 -str_to_key()-> 0000000000000000

③ 两组8字节数据将做为DESKEY对魔术字符串“KGS!@#$%”进行标准DES加密

"KGS!@#$%" -> 4B47532140232425

LISS
空天信息安全与可信计算
教育部重点实验室

Windows下LM Hash值生成原理

56A25288347A348A -对4B47532140232425进行标准DES加密-> C23413A8A1E7665F

0000000000000000 -对4B47532140232425进行标准DES加密-> AAD3B435B51404EE

④ 将加密后的这两组数据简单拼接，就得到了最后的LM Hash

LM Hash: C23413A8A1E7665FAAD3B435B51404EE

◆ LM Hash一般用于WINXP，Win2000系统中

Windows下NTLM Hash生成原理

① 假设明文口令是“123456”，首先转换成Unicode字符串，

"123456" -> 310032003300340035003600

② 对所获取的Unicode串进行标准MD4单向哈希，无论数据源有多少字节，MD4固定产生128-bit的哈希值，

16字节310032003300340035003600 -进行标准MD4单向哈希-> 32ED87BDB5FDC5E9CBA88547376818D4

③ 就得到了最后的NTLM Hash

NTLM Hash: 32ED87BDB5FDC5E9CBA88547376818D4

- Win7和Vista禁用了LM，默认使用NTLM
- WIN 10使用AES和MD5生成SAM密码

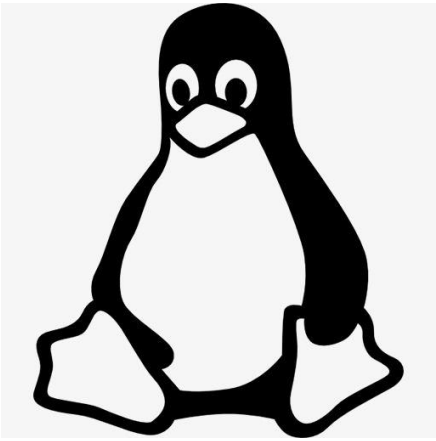
LISS
空天信息安全与可信计算
教育部重点实验室

Linux身份认证的实现机制

一、Linux的用户和组安全管理

1. Linux的用户

- 超级用户
  - 拥有操作系统的一切权限
  - 负责系统的启动、停止，用户管理
  - 用户 主目录：/root
  - UID值为0



Linux身份认证的实现机制

- 普通用户
  - 具有操作系统有限的权限通常情况下只能在自己的主目录下进行写操作
  - 主目录通常在/home下，其中包含用户的设置、文档、程序等
  - UID值500~6000
- 伪用户
  - 是为了方便系统管理，满足相应的系统进程对文件属主的的要求，伪用户不能登录
  - UID值1~499

Linux身份认证的实现机制

2. Linux的组

- 私有组(g)
  - 当在创建一个新用户user时，若没有指定他所属于的组，Linux就建立一个和该用户同名的私有组。
- 标准组 (G)
  - 标准组可以容纳多个用户，若使用标准组，在创建一个新的用户时就应该指定他所属于的组。

Linux身份认证的实现机制

用户账户文件：

- 口令文件

/etc/passwd

- 影子口令文件

/etc/shadow

-只有root用户对以上文件具有修改权限

Linux身份认证的实现机制

Passwd文件：

```
jiao@ubuntu:/etc$ cat passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
syslog:x:101:103::/home/syslog:/bin/false
```

Linux身份认证的实现机制

Passwd文件格式：

- pp : x : 500 : 500 : : /home/ pp : /bin / bash
- 账号名称：　　在系统中是唯一的
- 用户密码：　　此字段存放加密口令
- 用户标识码(User ID)：　系统内部用它来标示用户
- 组标识码(Group ID)：　系统内部用它来标识用户属性
- 用户相关信息：　　例如用户全名等
- 用户目录：　　用户登录系统后所进入的目录
- 用户环境:　　用户工作的环境，Linux默认为bash

Linux身份认证的实现机制

shadow文件：

```
jiao@ubuntu:/etc$ sudo cat shadow|more
root:!:15586:0:99999:7:::
daemon:*:15089:0:99999:7:::
bin:*:15089:0:99999:7:::
sys:*:15089:0:99999:7:::
sync:*:15089:0:99999:7:::
games:*:15089:0:99999:7:::
man:*:15089:0:99999:7:::
lp:*:15089:0:99999:7:::
mail:*:15089:0:99999:7:::
news:*:15089:0:99999:7:::
uucp:*:15089:0:99999:7:::
proxy:*:15089:0:99999:7:::
www-data:*:15089:0:99999:7:::
backup:*:15089:0:99999:7:::
jiao:$1$W$mRB5WjJLnlEE1yy7P/I3X.:15586:0:99999:7:::
mysql:!:15586:0:99999:7:::
openca:!:15586:::::
postfix:*:15586:0:99999:7:::
tss:!:15645:::::
jiao@ubuntu:/etc$
```

Linux身份认证的实现机制

shadow文件格式：

- shadow文件中每条记录用冒号间隔的9个字段组成
- 用户名：　用户登录到系统时使用的名字，而且是唯一的
- 口令：　存放加密的口令
- 最后一次修改时间：标识从某一时刻起到用户最后一次修改时间
- 密码不能被修改的天数：0表示随时可以修改
- 密码需要重新修改的天数：99999表示密码永远有效
- 警告时间：　从系统开始警告到口令正式失效的天数
- 失效时间：指示口令失效的绝对天数(从1970年1月1日开始计算

Linux身份认证的实现机制

Shadow文件格式

- 数字和所使用的加密算法对应关系：
- 1： MD5 ，（22位）
- 2: Blowfish, 只在有一部分linux分支中使用的加密方法
- 5: SHA-256 （43位）
- 6: SHA-512 （86位）

Linux身份认证的实现机制

- Hash散列存储口令
  计算hash(password)，然后存储到shadow文件
- Salt
  计算hash(salt,password)，然后存储到shadow文件.
  salt-通常是8-12位的随机数。

Linux身份认证的实现机制

口令加盐算法：

$$D_{salt} = \mathbf{A_{random}}(\,)$$

$$\mathbf{A_{salt}}(D_{salt},\ D_{pw})$$

(1) 把盐值附加到口令上：$D_{tmp} = D_{pw} \,||\, D_{salt}$；

(2) 生成哈希值：$D_{hash} = \mathbf{A_{hash}}(D_{tmp})$；

(3) 以 D_{hash} 作为返回结果。

Linux身份认证的实现机制

加盐的身份认证：口令字段维护

(1) 接收用户提供的口令 D_{pw};

(2) 生成一个盐值：$D_{salt} = \mathbf{A_{random}}(\ )$;

(3) 生成口令信息：$s = \mathbf{A_{gen}}(D_{salt}, D_{pw})$;

(4) 把口令信息 s 和 D_{salt} 存入数据库的口令字段中。

2、基于口令的身份认证

加盐的身份认证：身份认证

(1) 接收用户提供的帐户名 D_{name} 和口令 D_{pw}；

(2) 在帐户信息数据库中检查 D_{name} 的合法性，如果合法，则找出其对应的 s 和 D_{salt}；

(3) 生成临时口令信息：$s_r = \mathbf{A_{gen}}(D_{salt}, D_{pw})$；

(4) 如果 s_r 与 s 相等，则认证成功，否则，认证失败。

Linux身份认证的实现机制

Base64编码—介绍

Base64是一种传输编码；
是为了适应将任何计算机数据都表示成便于显示和排版的字符串；

标准Base64中的64个字符是：
“ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz+/0123456789”

Linux身份认证的实现机制

Base64编码—介绍

2的6次方等于64。
因为base64只有64个可用字符，所以它编码后的每个字节只能表示6个bit的信息内容；

我们待编码的内容是8个bit的，所以就造成了编码以后的数据比原来的数据要长；

6和8的最小公倍数是24，故base64的最小编码单元是24bit，四个字符。

LISS
空天信息安全与可信计算
教育部重点实验室

Linux身份认证的实现机制

Base64编码一举例

例如我们要对字符串“123”编码：

“123”对应的16进制数为：0x31 0x32 0x33；

表示成二进制数表示如下：

00110001 00110010 00110011；

将二进制数每6位分成一组表示如下：

001100 010011 001000 110011；

每6位分段后得到的四个数字分别是：12 19 8 51

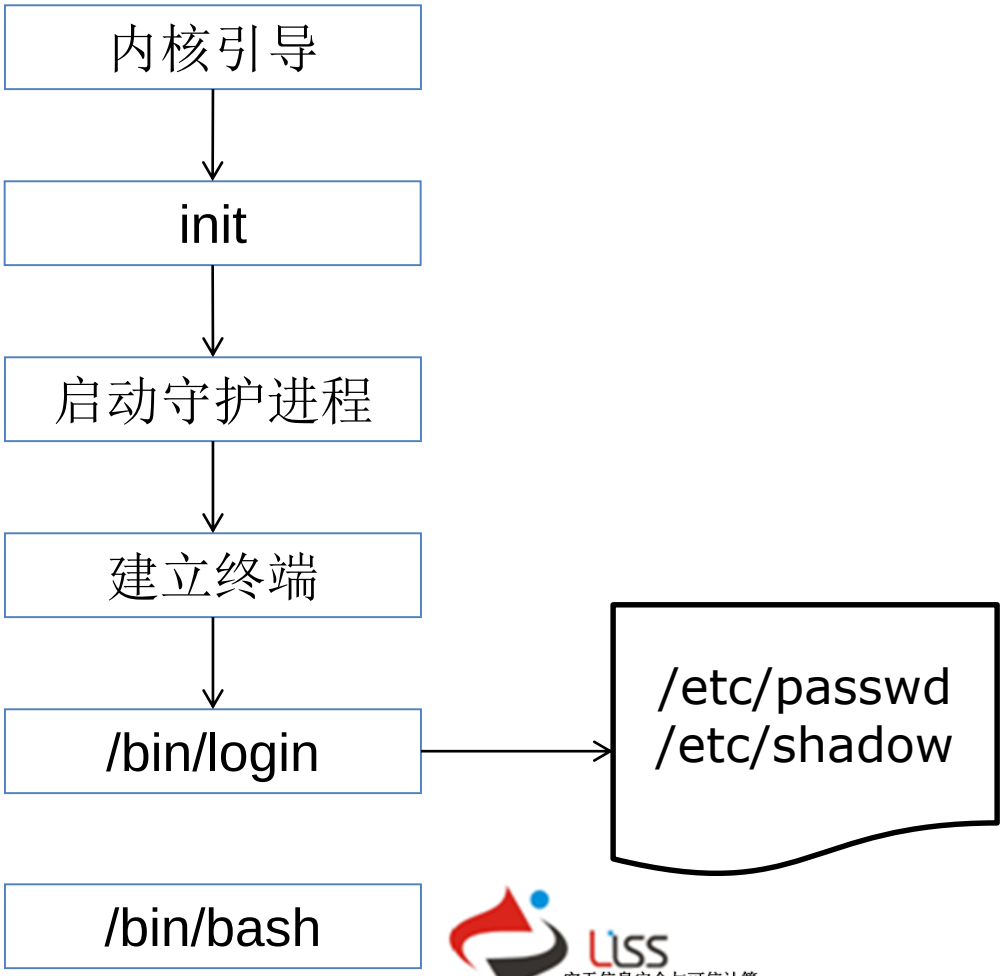
在base64编码表中查表对应的字符： M T I z

所以字符串”123”编码后是“MTIz”

LISS
空天信息安全与可信计算
教育部重点实验室

Linux身份认证的实现机制

Linux身份认证--login流程：



空天信息安全与可信计算
教育部重点实验室

口令破解

- 常用的口令破解工具：
  - Brutus
    - 目前最流行的远程在线密码破解工具
  - RainbowCrack
    - 基于彩虹表的密码破解工具，提供LM彩虹表、NTLM彩虹表和MD5彩虹表，支持Windows和Linux
  - John the Ripper
    - 开源密码破解工具，可以运行在Linux、Unix和Mac OSX上
  - Aircrack-NG
    - 一个WiFi密码破解工具，能够破解WEP或WPA密码。

LISS
空天信息安全与可信计算
教育部重点实验室

3、基于智能卡的身份认证

1、智能卡的安全性：

IC卡是Integrated Circuit（集成电路）卡的缩写，也称“MEMORY CARD”和“SMART CARD”，中文译作“聪明卡”、“智慧卡”。

它是一种将具有加密、存储、处理能力的集成电路芯片嵌装于塑料基片上而制成的卡片

3、基于智能卡的身份认证

2、基于智能卡的认证机制：

IC卡读卡器

应用程序与读卡器通信，而读卡器将使用上面介绍的标准与智能卡通信。在用户进行商务交易前，服务器首先使用智能卡完成用户身份的认证。

LISS
空天信息安全与可信计算
教育部重点实验室

3、基于智能卡的身份认证

身份认证过程中为了产生变动的密码一般采用双运算因子的计算方式，也就是加密算法的输入值有两个数值，其一为用户密钥、另一为变动因子，由于用户密钥为固定数值，因此变动因子必须不断变动才可以算出不断变动的动态密码。服务器及智能卡必须随时保持相同的变动因子，才能算出相同的动态密码。

3、基于智能卡的身份认证

(1)询问/应答认证:

变动因子是由服务器产生的随机数字。认证过程如下：

1）登录请求。客户机首先向服务器发出登录请求，服务器提示用户输入用户ID和PIN 。

3、基于智能卡的身份认证

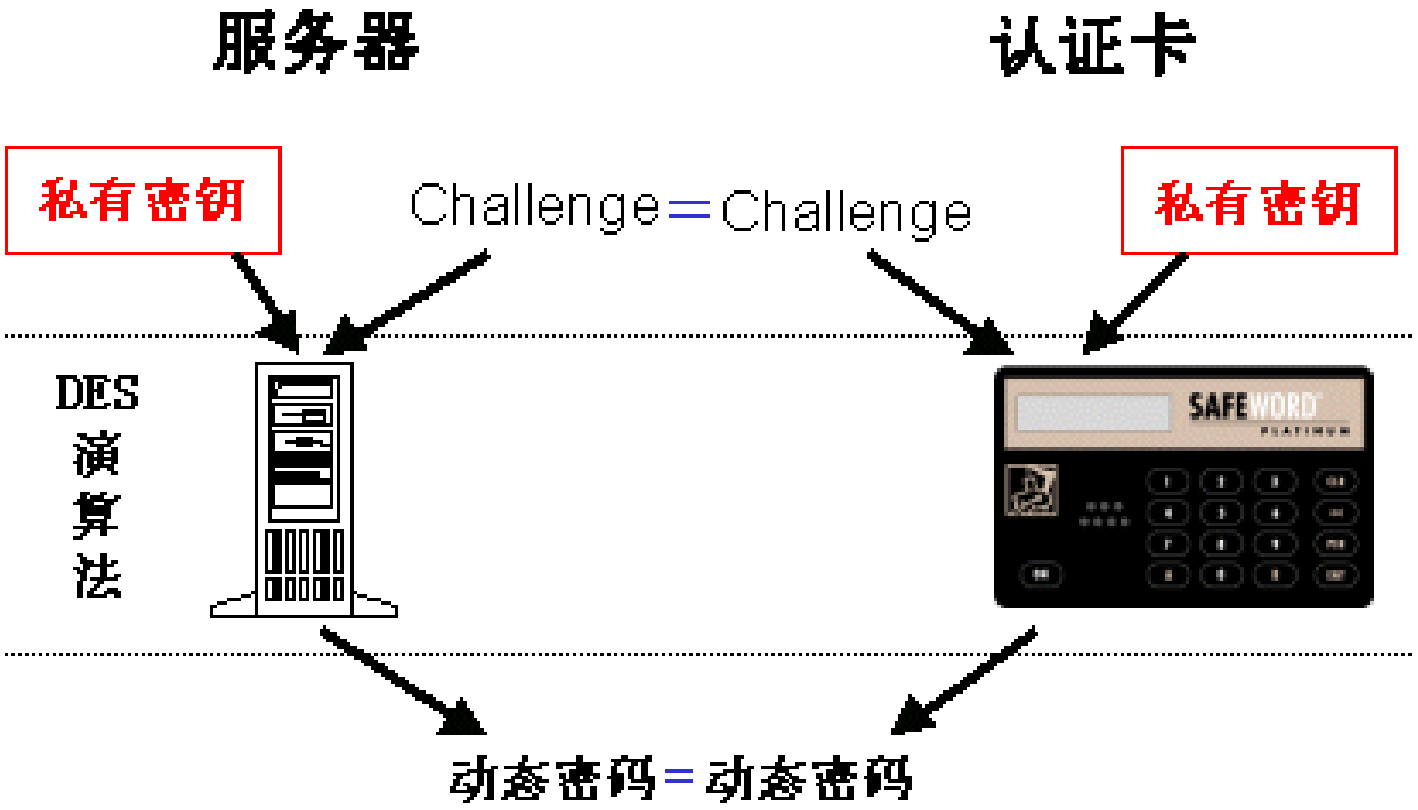
2）询问。用户提供ID给服务器，然后服务器提供一个随机串X（Challenge）给插在客户端的智能卡作为验证算法的输入，服务器则根据用户ID取出对应的密钥K后，利用发送给客户机的随机串X，在服务器上用加密引擎进行运算，得到运算结果RS。

3、基于智能卡的身份认证

3）应答。智能卡根据X与内在密钥K使用硬件加密引擎运算，也得到一个运算结果RC，并发送给服务器。

4）验证。比较RS和RC便可确定用户的合法性。

3、基于智能卡的身份认证



3、基于智能卡的身份认证

由于密钥存在于智能卡中，运算过程也是在智能卡中完成，密钥认证是通过加密算法来实现的，因而极大地提高了安全性。并且每当客户端有一次服务申请时，服务器便产生一个随机串给客户，即使在网上传输的认证数据被截获，也不能带来安全上的问题。

3、基于智能卡的身份认证

询问/应答身份认证的优点：

- 没有同步的问题。
- 一片认证卡可以用来存取被不同认证服务器所保护的系统。
- 最安全的认证方式。

缺点：

- 使用者必须按较多的按钮，操作较繁复。
- 比较多输入的失误。

3、基于智能卡的身份认证

2)时间同步认证

变动因子使用服务器端与客户端的同步时间值。认证过程如下：

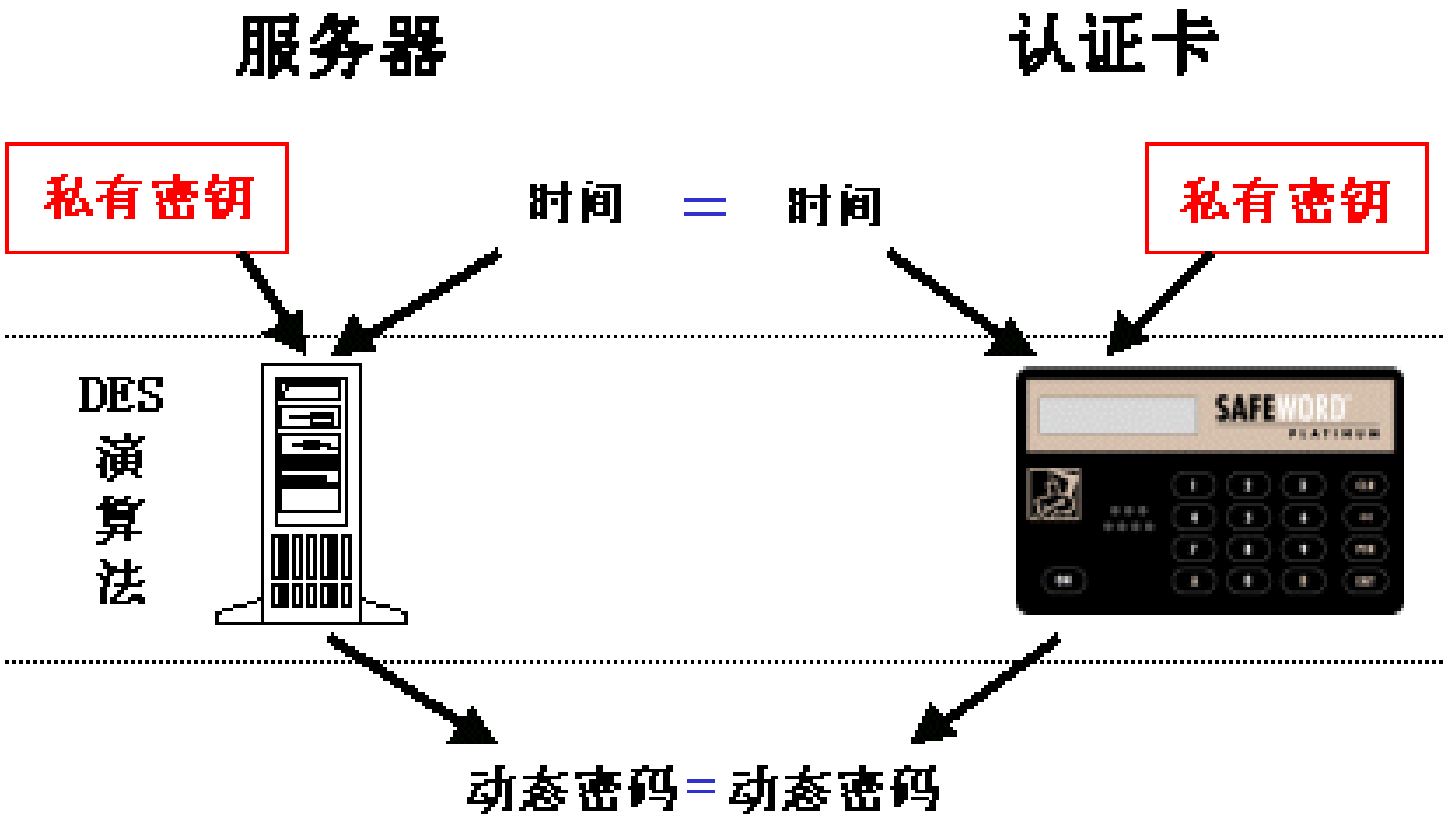
1）用户向服务器发出登录请求，服务器提示用户输入用户ID和用户PIN。

2）服务器根据用户ID取出对应的密钥K，使用K与服务器时间T计算动态密码RS。

3）智能卡根据内在的密钥K与客户机时间T使用相同的专用算法计算动态密码RC，并发送给服务器。

3、基于智能卡的身份认证

4）服务器比较RS与RC，如果相同则用户合法。



LISS
空天信息安全与可信计算
教育部重点实验室

3、基于智能卡的身份认证

时间同步认证的优点：

- 易于使用

缺点：

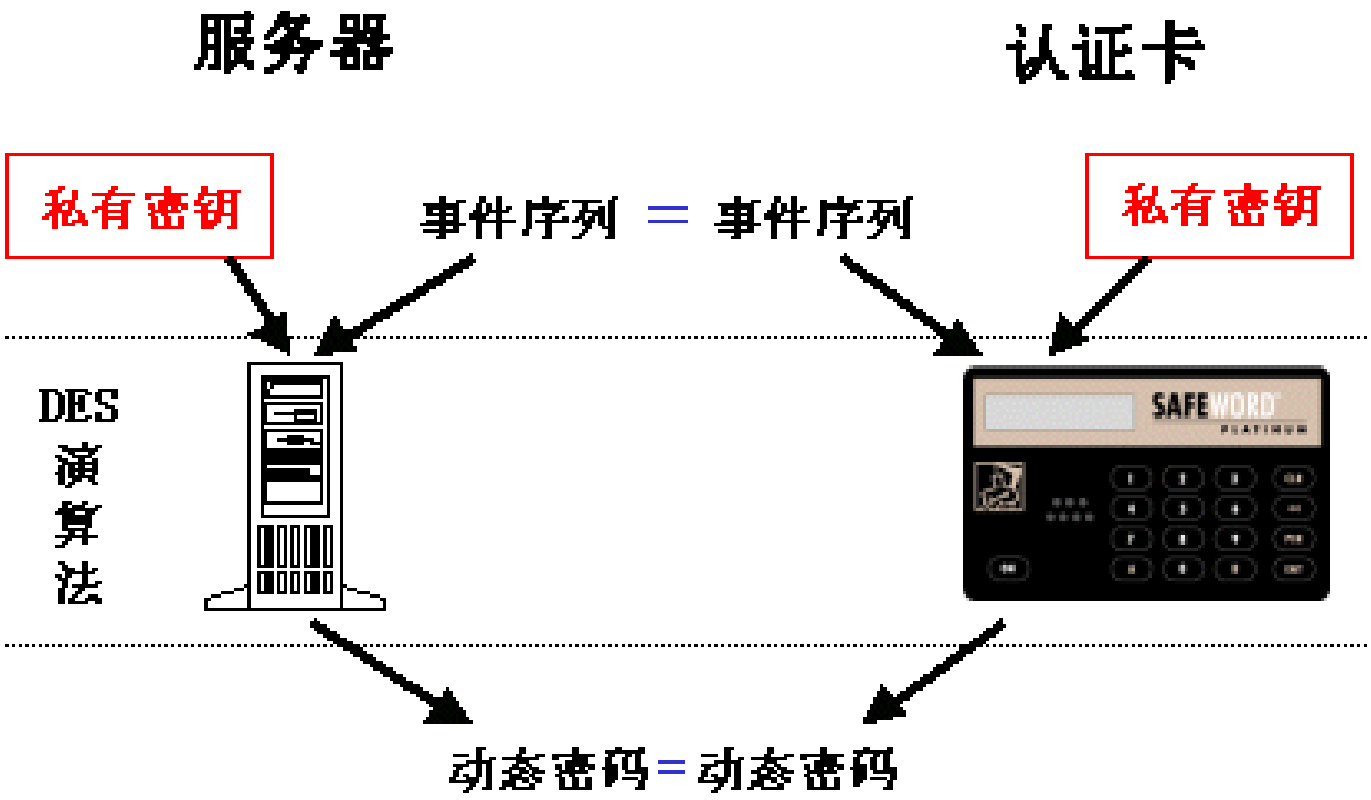
- 时间同步困难，可能造成必须重新输入新密码。软体认证卡采用PC的时刻，很可能随时被修改。常常需要与服务器重新对时。
- 不如Challenge/Response认证更安全

3、基于智能卡的身份认证

3)事件同步

事件同步认证卡依据认证卡上的私有密钥产生一序列的动态密码，如果使用者意外多产生了几组密码造成不同步的状态，服务器会自动重新同步到目前使用的密码，一旦一个密码被使用过后，在密码序列中所有这个密码之前的密码都会失效。

3、基于智能卡的身份认证



3、基于智能卡的身份认证

事件同步认证的优点：

- 容易使用。
- 由于使用者无法知道序列数字，所以安全性高，序列号码绝不会显示出来。

缺点：

- 如果没有PIN号码的保护及认证卡借给别人使用时，会有安全的疑虑。

4、局域网身份认证-Kerberos

Kerberos：为网络通信提供可信第三方服务的面向开放系统的认证机制。

来源于希腊神话“三个头的狗——地狱之门守护者”

4、局域网身份认证-Kerberos

Kerberos希望实现三个功能：身份认证、记账、审核。

Kerberos针对分布式环境，一些工作站可能安装于不安全场所，而且用户也并非是完全可信的。

4、局域网身份认证-Kerberos

Kerberos常用术语的简写：

- C：客户机
- AS：认证服务器
- S：应用服务器
- IDc、IDv、IDtgs 分别为C、V、TGS的身份
- ADc： 用户的网络地址
- TSi：第i个时戳

LISS
空天信息安全与可信计算
教育部重点实验室

4、局域网身份认证-Kerberos

- Lifetime： 第I个有效期限
- Pc：C上的用户口令
- Kc：C和AS的共享密钥
- Ks：S和TGS的共享密钥
- Ktgs： TGS 和AS的共享密钥
- Kc,tgs：C与TGS的共享密钥
- Kc,s：C与S的共享密钥

4、局域网身份认证-Kerberos

开放式系统的认证的要求：

1) 安全性：没有攻击的薄弱环节。

2) 可靠性：认证服务是其他服务的基础，要可靠，不能瘫痪。

3) 透明性：用户觉察不到认证服务，只是输入口令。

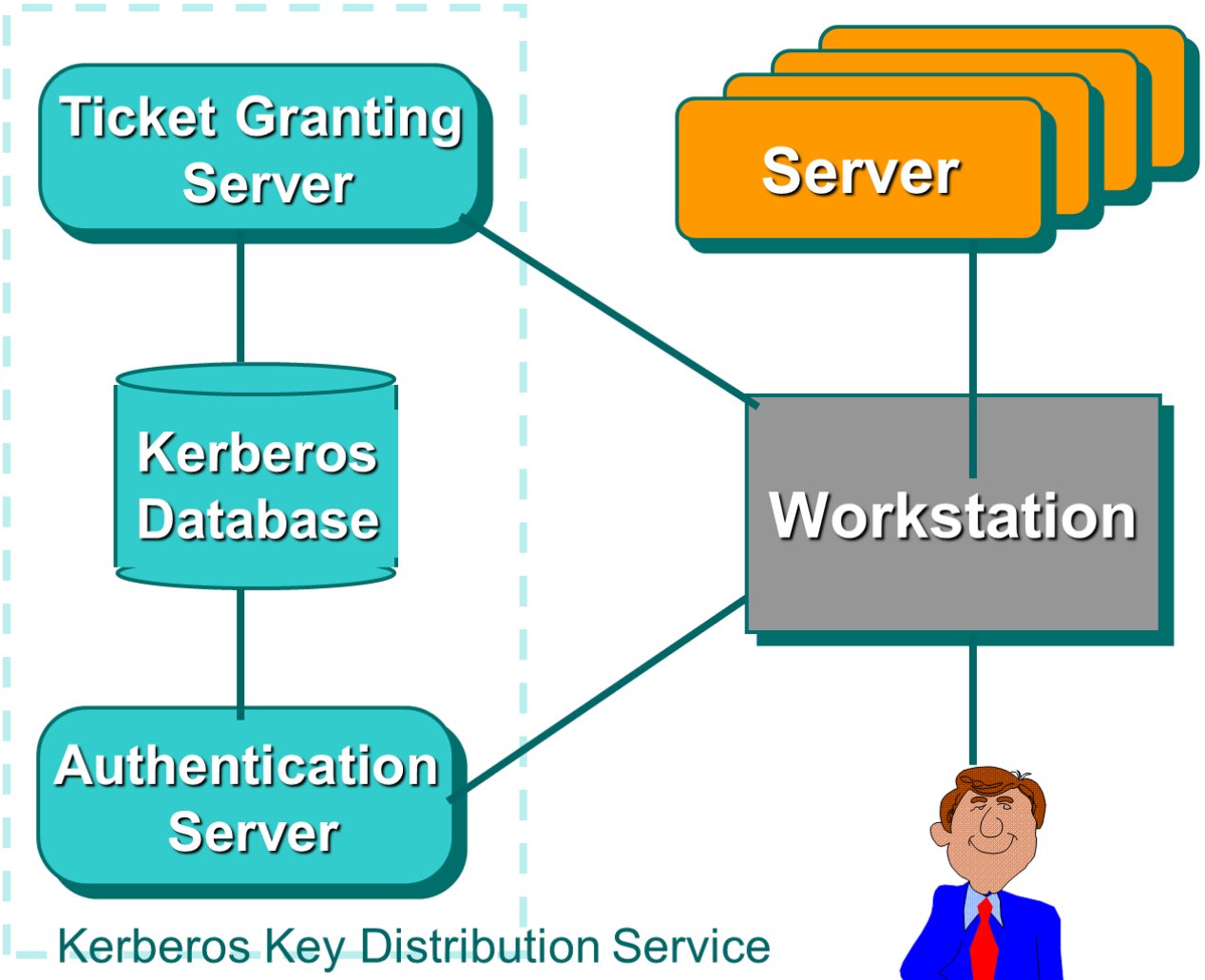
4) 可扩展性：支持加入更多的服务器。

4、局域网身份认证-Kerberos

Kerberos的认证模型如下：

- 认证服务器AS：它同时应该连接并维护一个中央数据库存放用户口令、标识等）
- 票据许可服务器TGS(Ticket Granting Server)
- Client
- Server

4、局域网身份认证-Kerberos



LISS
空天信息安全与可信计算
教育部重点实验室

4、局域网身份认证-Kerberos

Kerberos 有两种证书：票据ticket和认证符authenticator。这两种证书都要使用对称加密算法进行加密，但加密的密钥不同。

Ticket用来安全地在AS和S之间传递用户的身份，同时保证使用ticket的用户必须是ticket中指定的用户。

4、局域网身份认证-Kerberos

Ticket的组成：

- C/S的标识
- client的地址
- 时间戳
- 生存时间
- 会话密钥

Ticket一旦生成，在life指定的时间内可以被client多次使用来申请同一个server的服务。

4、局域网身份认证-Kerberos

Ticket的种类：

- $Ticket_{tgs}$
  - 组成：用户名，用户地址，TGS服务名，当前时间，有效时间，会话密钥Kc,tgs
  - 作用：AS颁发，用于TGS服务器验证用户的身份
- $Ticket_{s}$
  - 组成：C和S的名字，C的网络地址，当前时间，有效时间和会话密钥Kc,s
  - 作用：TGS颁发，用于应用服务器确定用户身份

LISS
空天信息安全与可信计算
教育部重点实验室

4、局域网身份认证-Kerberos

Authenticator：提供信息与ticket中的信息进行比较，一起保证发出ticket的用户就是ticket中指定的用户。

4、局域网身份认证-Kerberos

认证符的组成：

- client的名字
- client的地址
- 记录当前时间的时间戳

authenticator只能在一次服务请求中使用，每当client向server申请服务时，必须重新生成Authenticator。

LISS
空天信息安全与可信计算
教育部重点实验室

4、局域网身份认证-Kerberos

用户C请求服务S的整个Kerberos认证过程：

1)认证服务交换

用户C向AS发出请求，以获取访问TGS的令牌(票据许可证)：$Ticket_{tgs}$

① Client->AS

KRB_AS_REQ={c, tgs, TS1}

c表示Client客户标识，告诉AS请求的Client身份，便于AS到数据库中查询c的密钥

tgs是Client下一步要访问TGS服务器标识

TS1时间戳被AS用于验证Client时钟是否与AS时钟同步

4、局域网身份认证-Kerberos

用户C请求服务S的整个Kerberos认证过程：

1)认证服务交换

用户C向AS发出请求，以获取访问TGS的令牌(票据许可证)：$Ticket_{tgs}$

② AS->Client

KRB_AS_REP={K(c,tgs),TS2,lifetime,tgs_Tickets}Kc

K(c,tgs)是由AS随机产生的，用于Client与TGS共享的一个会话私钥

TS2是票据发放的时间戳，lifetime是票据有效期

tgs_Tickets是AS发给Client的用于访问TGS的票据授权票据

tgs_Tickets={K(c,tgs), c, c_mac, tgs, TS2, lifetime2}Ktgs

c_mac是Client的MAC地址

4、局域网身份认证-Kerberos

2) C从TGS得到所请求服务的令牌Ticket$_V$。

一个令牌只能申请一个特定的服务，所以用户必须为每一个服务V申请新的令牌，用户可以从TGS处得到服务令牌Ticket$_V$ 。

③ Client->TGS

KRB_TGS_REQ={s, tgs_Tickets, Authen_info}

s是Client期望访问的AppServer服务器

tgs_Tickets是Client上一步从AS获取的票据授权票据

Authen_info是Client发送的一条认证消息，用于确认发送票据tgs_Tickets的用户就是c

Authen_info={c, c_mac, TS3}K(c,tgs)

4、局域网身份认证-Kerberos

2) C从TGS得到所请求服务的令牌Ticket$_V$。

④ TGS->Client

KRB_TGS_REP={K(c,s), s, TS4, s_Tickets}K(c,tgs)

K(c,s)是TGS随机生成的密钥，为Client端和AppServer共享的会话密钥

s_Tickets={K(c,s), c, c_mac, s, TS4, lifetime}Ks

TGS用自己的密码Ktgs解密tgs_Tickets获得

c,c_mac,K(c,tgs),TS2,lifetime

TGS首先根据时TS2,lifetime时间判断票据是否过期，未过期则继续，

TGS再用上面解密出来的会话私钥K(c,tgs)解密Authen_info，获得c,c_mac，

比较上面2次解密出来的c,c_mac是否一致，判断此次请求来自合法Client。

4、局域网身份认证-Kerberos

(3) 客户机与应用服务器之间认证交换。

C向应用服务器发送服务许可票据 $Ticket_s$ 和认证符Authenticator_info

⑤ Client->AppServer
KRB_APPSERVER_REQ={s_Tickets,Authent_info}，
Client使用K(c,tgs)从KRB_TGS_REP响应中获取K(c,s)
Client采用会话密钥K(c,s)加密Authent_info消息
Authen_info={c, c_mac, TS5}K(c,s)

LISS
空天信息安全与可信计算
教育部重点实验室

4、局域网身份认证-Kerberos

(3) 客户机与应用服务器之间认证交换。

⑥ AppServer->Client

KRB_APPSERVER_REP={TS5+1}K(c,s)

AppServer用自己的密码Ks从S_Tickets解密获得K(c,s), c, c_mac, s, TS4, lifetime

AppServer然后用TS4时间戳和有效期lifetime来检查票是否有效,有效才进行下一步

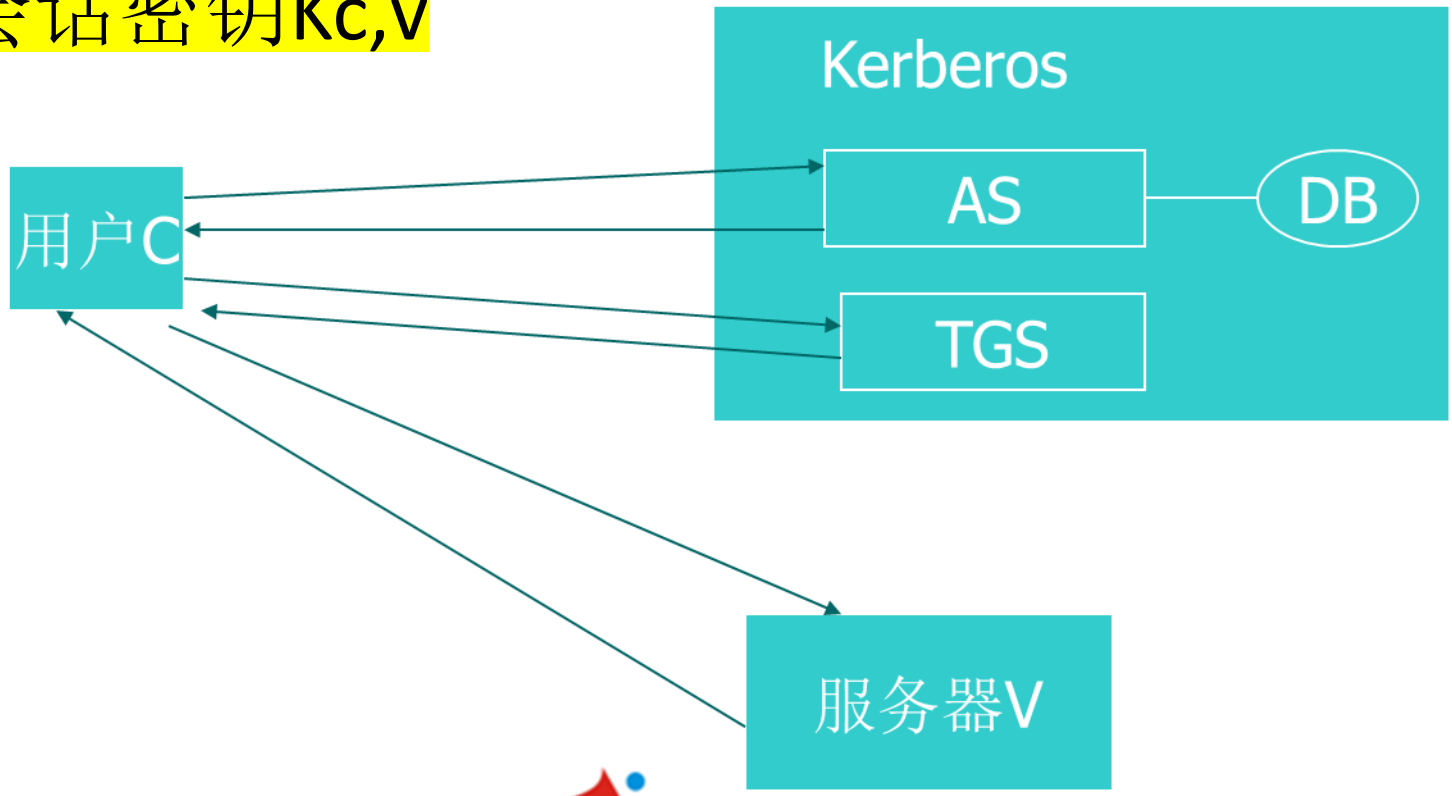
AppServer再用从上面解密出来的K(c,s)从Authen_info解密获得c, c_mac

AppServer比较上面2次解密出来的c,c_mac是否一致，判断此次请求来自合法Client。

最后AppServer使用K(c,s)加密确认消息给Client端，告知对方它期待的服务器已收到它的服务请求。

4、局域网身份认证-Kerberos

通过三阶段六步操作，用户C和服务V互相验证彼此的身份，并且拥有只有C和V两者知道的会话密钥Kc,v



LISS
空天信息安全与可信计算
教育部重点实验室

4、局域网身份认证-Kerberos

Kerberos V5的改进：

- 加密算法的多样化
- 票据有效期延长

  2^8*5分钟→任意
- 允许认证转发

  C可让V1代表自己访问V2
- 多区域认证关系简化

4、局域网身份认证-Kerberos

Kerberos的局限性：

- 时间同步问题
- 口令猜测问题
- 认证域之间的信任问题
- 重放攻击的问题
- 密钥的存储问题
- 系统程序的安全性、完整性问题

5、统一身份认证-PAM

PAM的概念：

Linux-PAM(Pluggable Authentication Modules）(基于Linux的插入式验证模块)是一组共享库，使用这些模块，系统管理者可以自由选择应用程序使用的验证机制。

也就是说，勿需重新编译应用程序就可以切换应用程序使用的验证机制。甚至，不必触动应用程序就可以完全升级系统使用的验证机制。

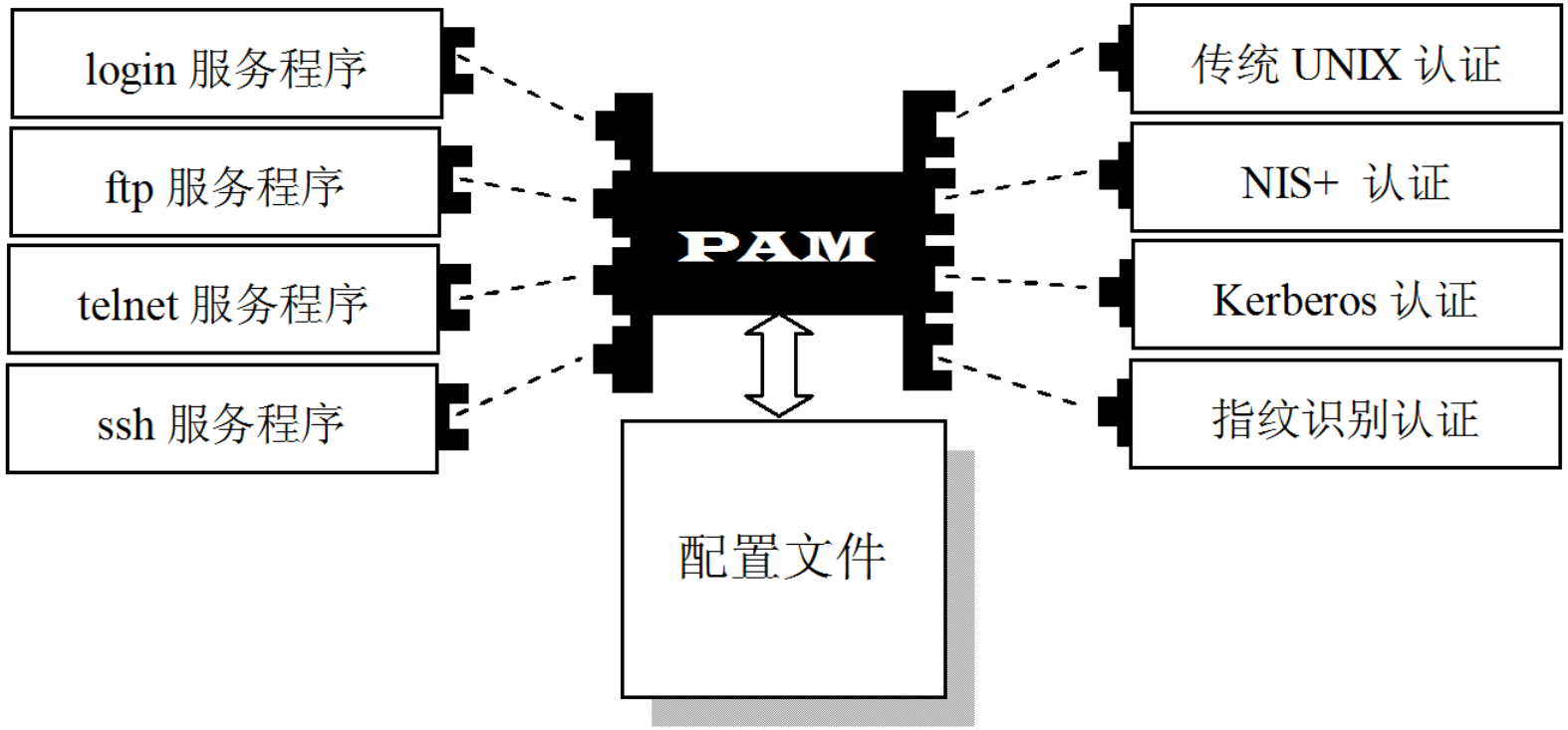
5、统一身份认证-PAM

Linux-PAM工程的目的:

分离应用软件和验证机制的开发。通过验证函数库可以实现上述目的。PAM库由本地的系统配置文件/etc/pam.conf或者/etc/pam.d/目录下的一些配置文件来设置。而模块以动态可加载目标文件的形式保存在/usr/lib/security目录中。

5、统一身份认证-PAM

PAM的架构：



5、统一身份认证-PAM

PAM认证系统的构成：

- PAM应用编程接口（API）
- PAM模块（动态装载库），提供以下服务功能支持：
  - 身份认证（auth）
  - 账户管理（account）
  - 口令管理（password）
  - 会话管理（session）
- PAM配置文件

5、统一身份认证-PAM

对于需要使用PAM的程序，需要在/etc/pam.d目录中为其设置配置文件。

配置文件内容和语法如下：

- service-name
- module-type
- control-flag
- module-path
- arguments

LISS
空天信息安全与可信计算
教育部重点实验室

5、统一身份认证-PAM

- Service-name

为这个入口分配的服务名。通常这是给定应用程序的会话名。例如：ftpd、rlogind、su等等。

- modle-type

Linux-PAM当前有四种类型的模块

auth account session password

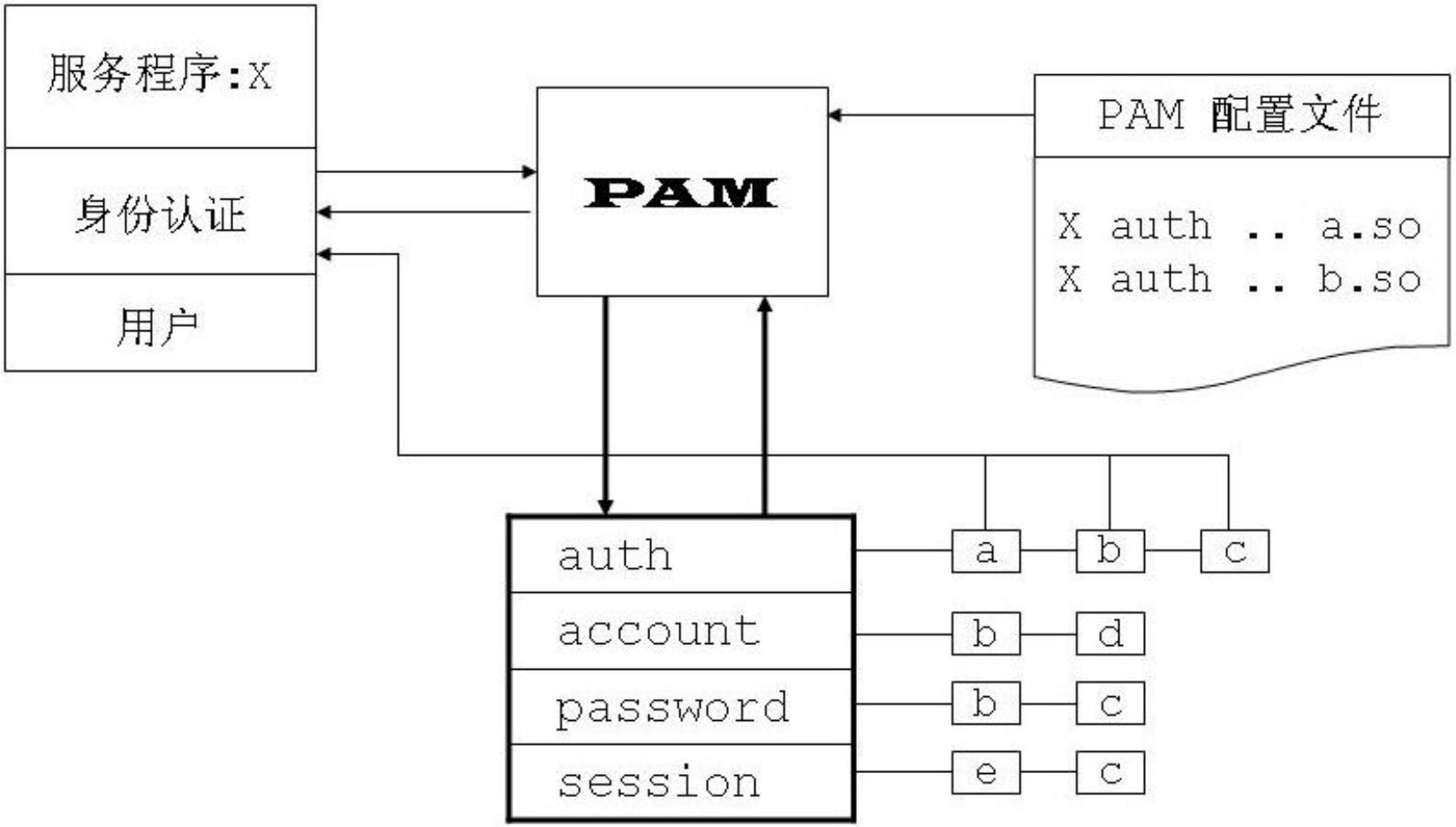
- control-flag

required requisite sufficient optional

LISS
空天信息安全与可信计算
教育部重点实验室

5、统一身份认证-PAM

PAM的工作原理：



5、统一身份认证-PAM

其他常见功能模块：

- pam_securetty

该模块用来控制root用户只可以从包含在/etc/securetty文件中的终端登录系统。

- pam_shell认证模块

如果用户的shell在/etc/shells中列出，则允许用户进行验证，如果/etc/passwd中没有指定shell，则缺省使用/bin/sh

5、统一身份认证-PAM

功能设置：

PAM加固：

- 提供基于登录名、主机名或域名、公网IP地址或网络号的登录设置
- 提供密码复杂性设置
- 密码强度检验的等功能
  - 例如：
    - /etc/pam.d/sshd 限制用户的登陆IP
    - /etc/pam.d/su 阻止任何人能够su作为root
    - /etc/pam.d/sshd 不允许所有用户登陆系统

本章总结

- 身份认证概述
- 基于口令的身份认证
  - Windows身份认证机制
  - Liunx身份认证机制
  - 口令破解
- 基于智能卡的身份认证
- 局域网身份认证—Kerberos
- 统一身份认证— PAM

思考题

- Linux和Windows是采用何种机制保存密码的？
- 什么是彩虹表？
- 如何把PPT中基于智能卡的身份认证方式扩展到基于公私钥方式？
- 为什么不让用户直接到应用服务器认证？Kerberos的优势在哪里？
- Kerberos中的两种票据，$Ticket_{tgs}$和$Ticket_{s}$的作用是什么？

问题？

谢　谢！

LISS
空天信息安全与可信计算
教育部重点实验室